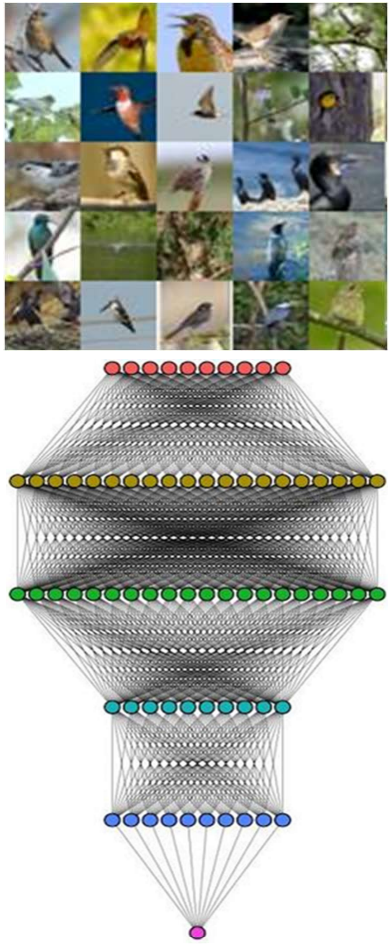
Perceptron

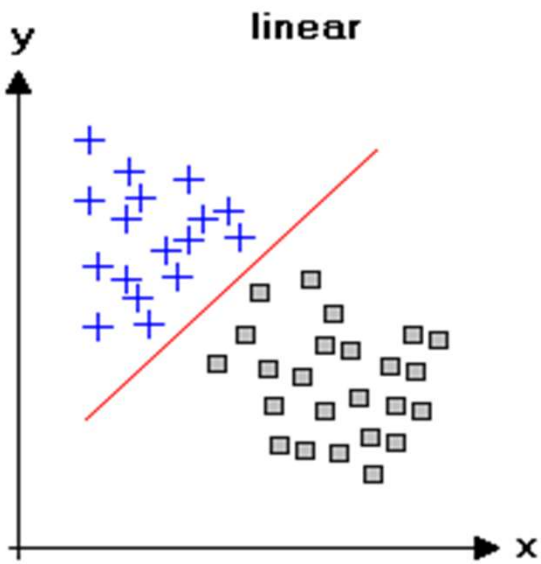
Prithwijit Guha
Dept. of EEE, IIT Guwahati

Supervised Learning

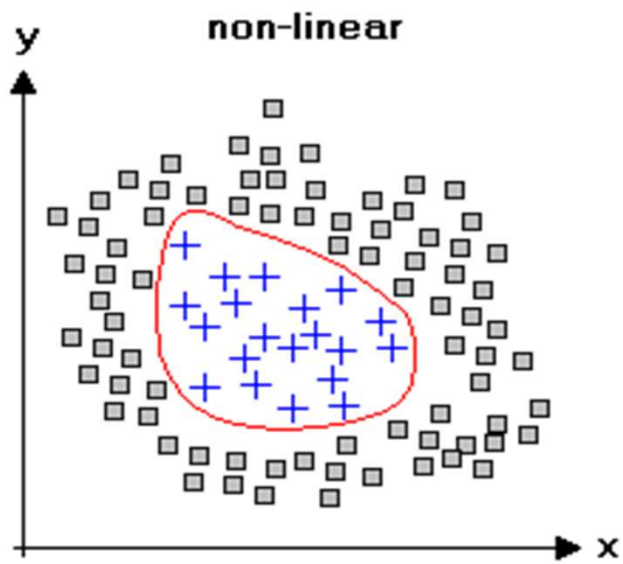


- Bayesian Classification, MAP, Chebyshev Inequality
- Performance Measures, Confusion Matrix, ROC Curves
- Discriminant Functions, LDA and Logistic Regression
- Perceptron
- Multi-Layer Perceptron (MLP), ELM
- MLP Architectures, Learning, Interpretations
- Non-parametric Methods and K-NN
- Radial Basis Function Neural Networks
- Data Balancing; SMOTE & Weighted Loss Functions
- Classification & Regression Trees
- Support Vector Machines & Multiple Kernel Learning
- Ensemble Methods, Bagging and Boosting

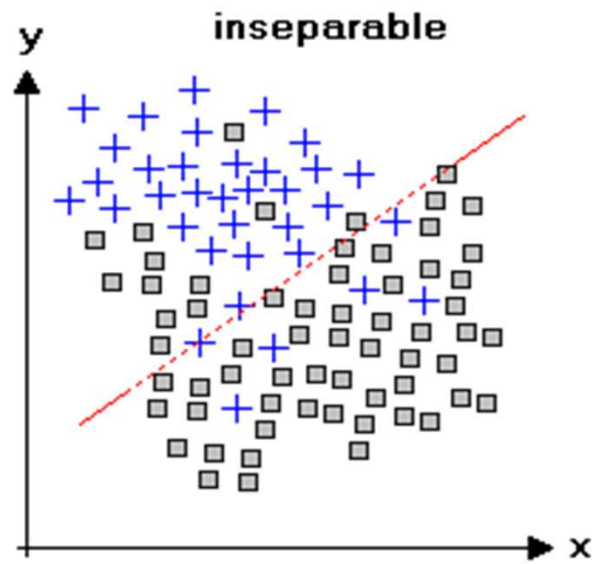
Separability in Feature Space



Hyperplanes Can Separate Features of Instances from Two Classes



Non-Linear Hyper-Surfaces Separate Features of Instances from Two Classes



Require Further Non-Linear Transforms for Separability in Transformed Space

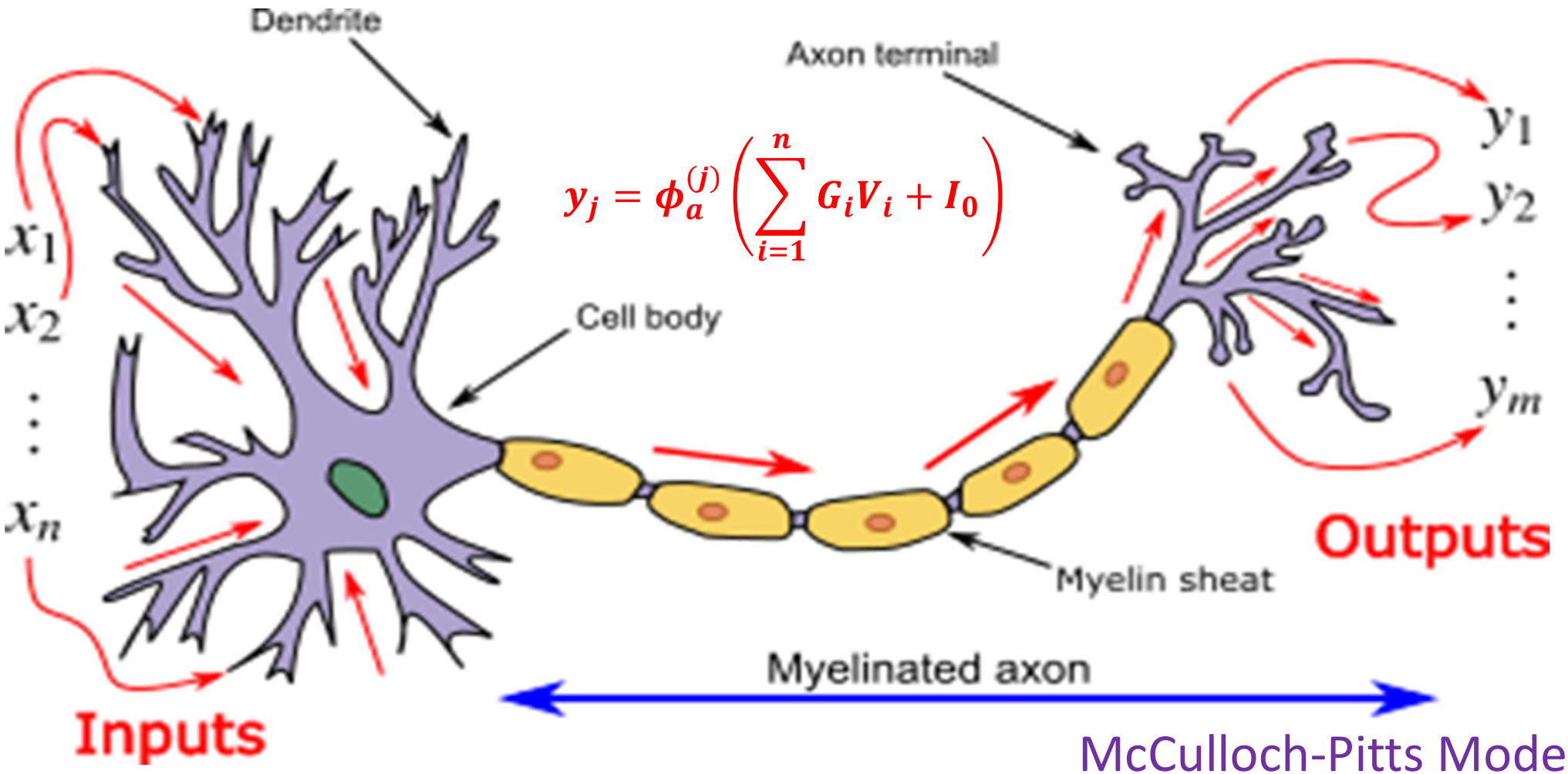
Binary Classification: Input-Output Specification

$$\boldsymbol{S_b} = \{(\boldsymbol{x}_i, y_i); i = 1, \dots n\}$$

$$\boldsymbol{x} \in \mathbb{R}^d \qquad\qquad y \in \{0,1\}$$

$$\boldsymbol{x} \longrightarrow \boxed{\text{Classifier } \boldsymbol{C_b}} \longrightarrow \hat{y} = \boldsymbol{C_b}(\boldsymbol{x})$$

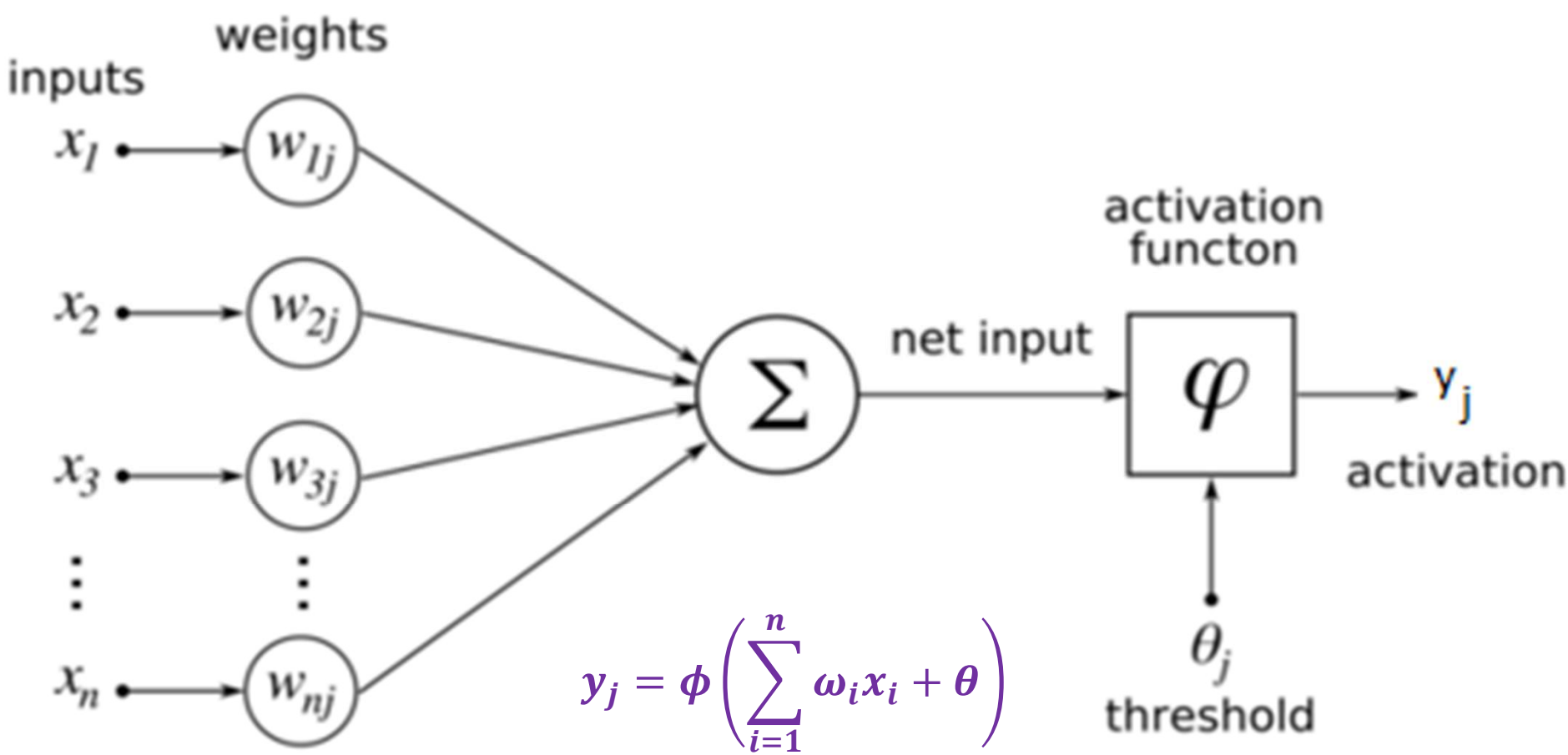
Perceptron: Biological Motivation



$$y_j = \phi_a^{(j)}\left(\sum_{i=1}^{n} G_i V_i + I_0\right)$$

McCulloch-Pitts Model

Perceptron: Mathematical Formulation



$$y_j = \phi\left(\sum_{i=1}^{n} \omega_i x_i + \theta\right)$$

Perceptron: The Model

$$\hat{y} = P(\boldsymbol{x}; \boldsymbol{\omega}, b)$$

$$P : \mathbb{R}^d \to (0,1)$$

Perceptron as a Function

$$u = \boldsymbol{\omega}^T \boldsymbol{x} + b = \sum_{j=1}^{d} \boldsymbol{\omega}_j \boldsymbol{x}_j + b$$

$\boldsymbol{\omega}[j]$ $\boldsymbol{x}[j]$

$$\hat{y}_i = \phi(u)$$

Linear Regression

$$u = \boldsymbol{\omega}^T \boldsymbol{x} + b$$

$$\hat{y} = \phi_{lin}(u) = u$$

Logistic Regression

$$u = \boldsymbol{\omega}^T \boldsymbol{x} + b$$

$$\hat{y} = \phi_{log}(u) = \frac{1}{1 + e^{-u}}$$

Perceptron as Classifier

$$\boldsymbol{S_b} = \{(\boldsymbol{x}_i, y_i); i = 1, \dots n\}$$

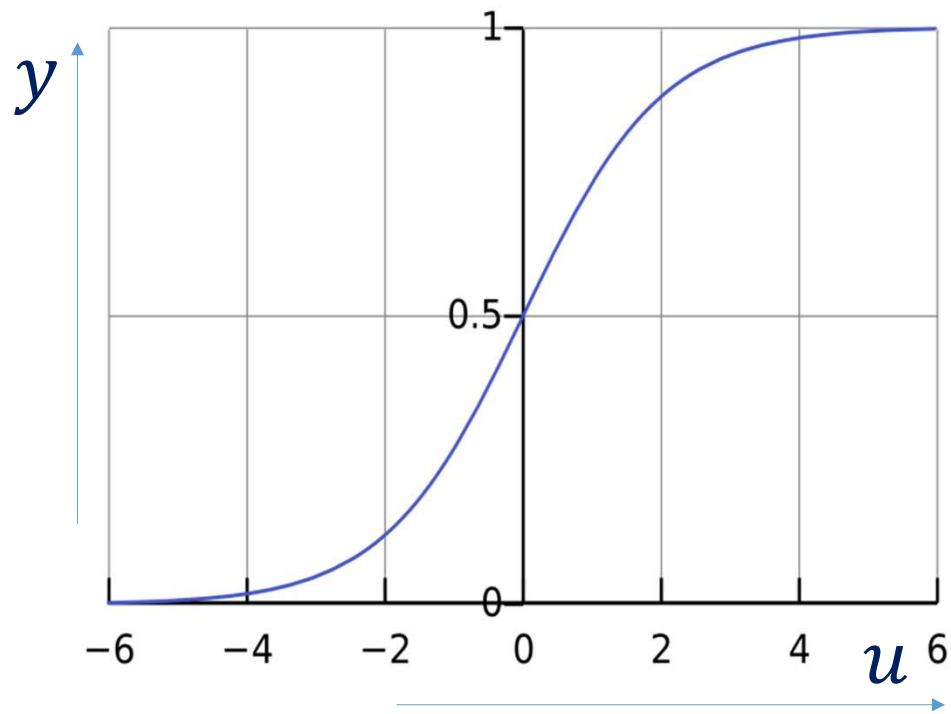
$$u_i = \boldsymbol{\omega}^T \boldsymbol{x}_i + b$$

$$u_i = \sum_{j=1}^{d} \boldsymbol{\omega}_j \boldsymbol{x}_{ij} + b$$

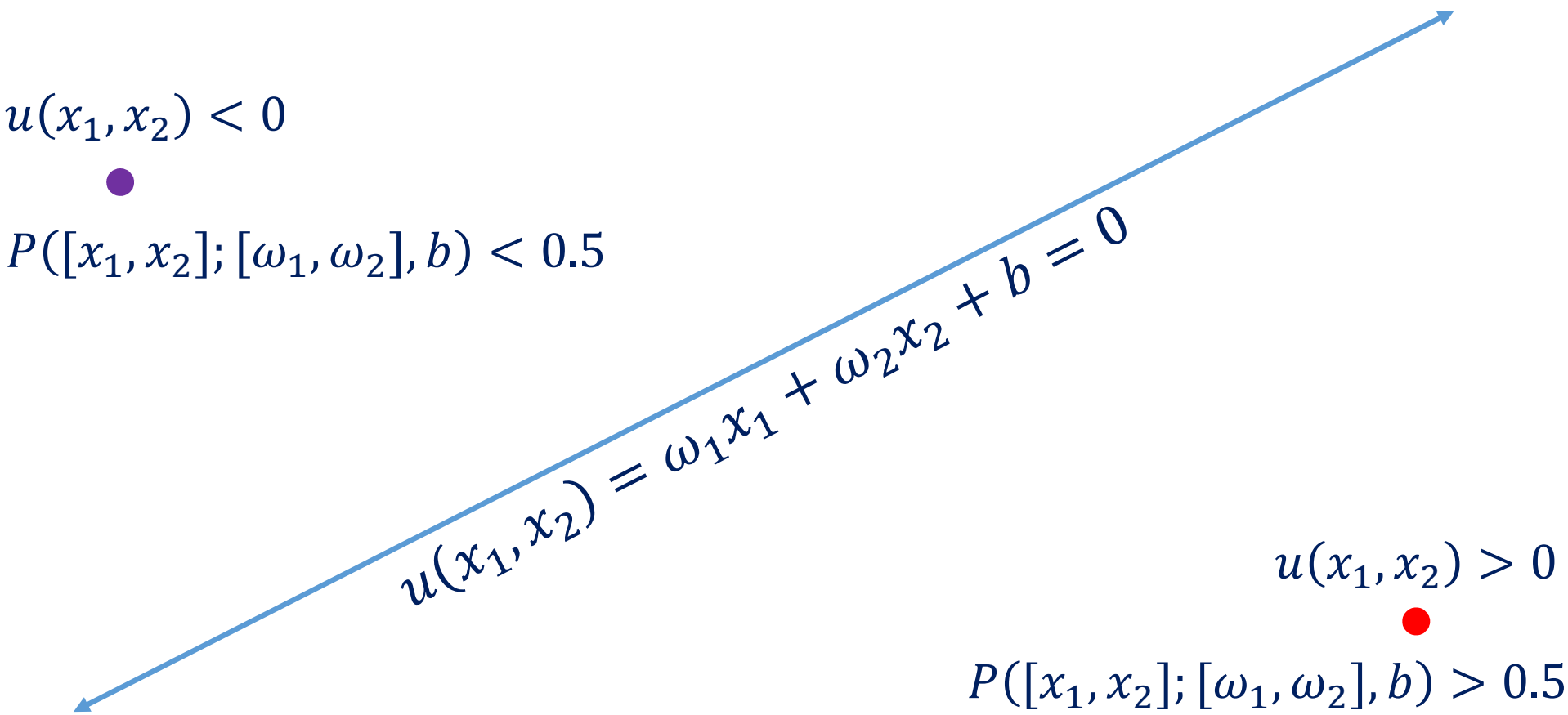
$$\hat{y}_i = \frac{1}{1 + e^{-u_i}}$$

$$\hat{y} = P(\boldsymbol{x}; \boldsymbol{\omega}, b)$$

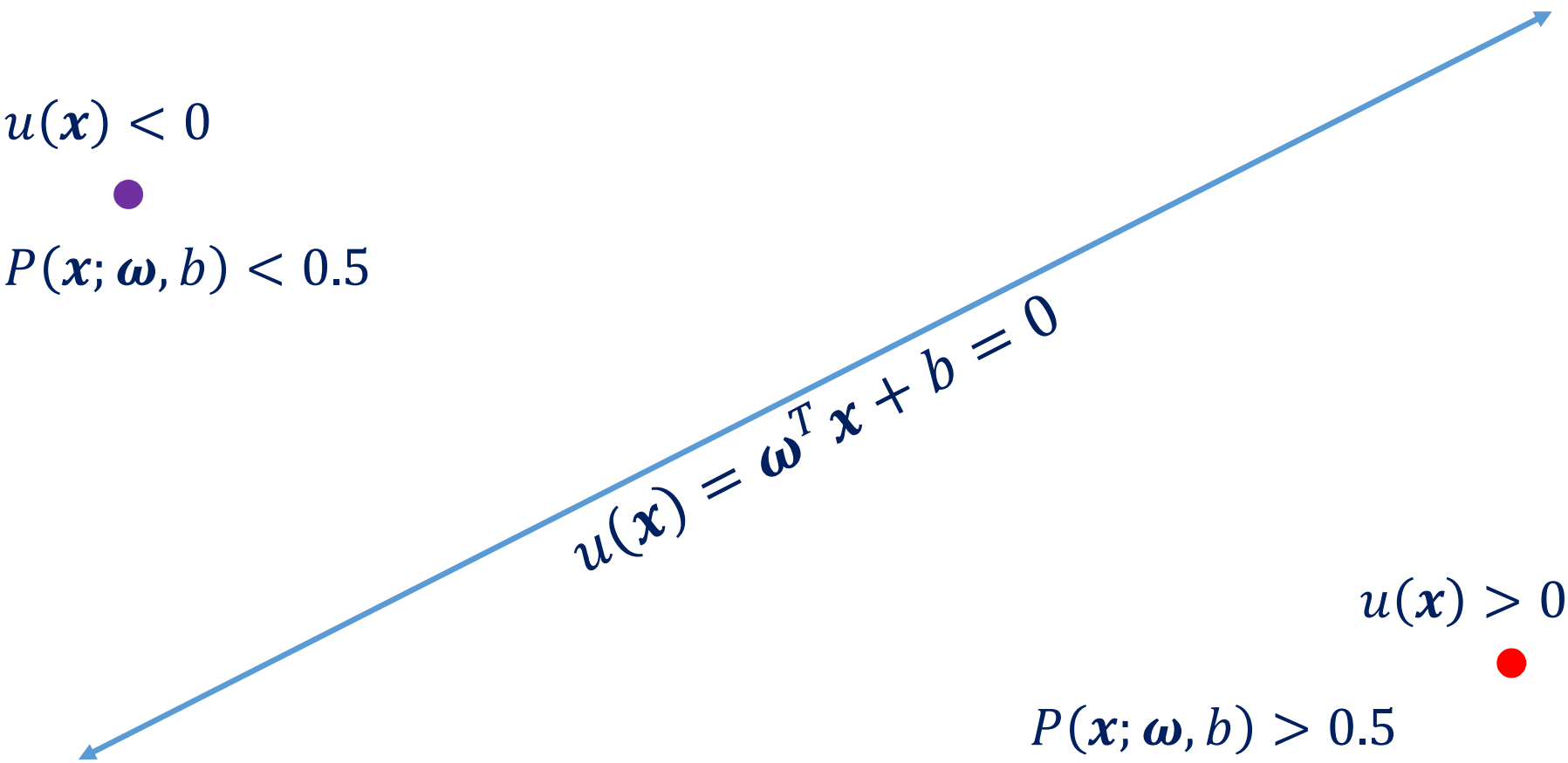
$$P : \mathbb{R}^d \to (0,1)$$



Perceptron: Linear Algebra Interpretation



Perceptron: Linear Algebra Interpretation

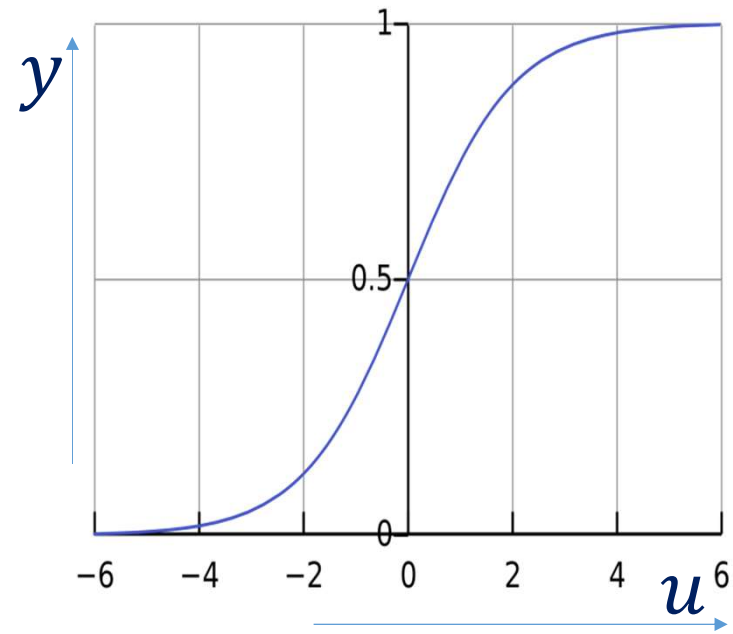


Perceptron as Feature Extractor

$$\hat{y} = P(\boldsymbol{x}; \boldsymbol{\omega}, b)$$

$$P : \mathbb{R}^d \to (0,1)$$

Perceptron as a Function



$$u = \boldsymbol{\omega}^T \boldsymbol{x} + b = \langle \boldsymbol{\omega}, \boldsymbol{x} \rangle + b$$

Inner Product

How Aligned (or Misaligned) are Patterns $\boldsymbol{\omega}$ and $\boldsymbol{x}$

Alignment Tolerance

$$\hat{y} = \frac{1}{1 + e^{-u}}$$

Perceptron: Learning by Gradient Descent

Dataset: $\boldsymbol{S_b} = \{(\boldsymbol{x}_i, y_i); i = 1, \dots n\}$

Instance Error: $e_i = y_i - \hat{y}_i = y_i - P(\boldsymbol{x}_i; \omega, b)$

Minimize Total Error: $E = \frac{1}{n}\sum_{i=1}^{n} e_i^2$

$$\omega_j^{(k+1)} = \omega_j^{(k)} - \eta_{jk}\frac{\partial E}{\partial \boldsymbol{\omega}_j}$$

$j = 1, \dots d$

$$b^{(k+1)} = b^{(k)} - \eta_{jk}\frac{\partial E}{\partial b}$$

Perceptron: Learning by Gradient Descent

Dataset: $\boldsymbol{S_b} = \{(\boldsymbol{x}_i, y_i); i = 1, \dots n\}$

Instance Error: $e_i = y_i - \hat{y}_i = y_i - P(\boldsymbol{x}_i; \omega, b)$

Minimize Total Error: $E = \frac{1}{n}\sum_{i=1}^{n} e_i^2$

$$\frac{\partial E}{\partial \boldsymbol{\omega}_j} = ?$$

$j = 1, \dots d$

$$\frac{\partial E}{\partial b} = ?$$

Assume Sigmoid Activation Function at Output

Perceptron: Learning by Gradient Descent

$$E = \frac{1}{n}\sum_{i=1}^{n} e_i^2 \quad \Rightarrow \quad \frac{\partial E}{\partial \boldsymbol{\omega}_j} = \frac{1}{n}\sum_{i=1}^{n} 2e_i \frac{\partial e_i}{\partial \boldsymbol{\omega}_j}$$

$$e_i = y_i - \hat{y}_i \quad \Rightarrow \quad \frac{\partial e_i}{\partial \boldsymbol{\omega}_j} = -\frac{\partial \hat{y}_i}{\partial \boldsymbol{\omega}_j}$$

$$\frac{\partial \hat{y}_i}{\partial \boldsymbol{\omega}_j} = \frac{\partial \hat{y}_i}{\partial u_i} \times \frac{\partial u_i}{\partial \boldsymbol{\omega}_j}$$

Perceptron: Learning by Gradient Descent

$$\hat{y}_i = \frac{1}{1 + e^{-u_i}} \quad \Rightarrow \quad \frac{\partial \hat{y}_i}{\partial u_i} = \hat{y}_i(1 - \hat{y}_i)$$

$$u_i = \sum_{r=1}^{d} \boldsymbol{\omega}_r \boldsymbol{x}_{ir} + b \quad \Rightarrow \quad \frac{\partial u_i}{\partial \boldsymbol{\omega}_j} = \boldsymbol{x}_{ij}$$

$$\frac{\partial E}{\partial \boldsymbol{\omega}_j} = \frac{1}{n} \sum_{i=1}^{n} 2e_i \frac{\partial e_i}{\partial \hat{y}_i} \times \frac{\partial \hat{y}_i}{\partial u_i} \times \frac{\partial u_i}{\partial \boldsymbol{\omega}_j}$$

Perceptron: Learning by Gradient Descent

$$\frac{\partial E}{\partial \boldsymbol{\omega}_j} = -\frac{2}{n}\sum_{i=1}^{n} e_i \hat{y}_i (1 - \hat{y}_i) \boldsymbol{x}_{ij}$$

$$\frac{\partial E}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n} e_i \hat{y}_i (1 - \hat{y}_i)$$

Perceptron: Learning by Pseudo-Inverse

$$\boldsymbol{S_b} = \{(\boldsymbol{x}_i, y_i); i = 1, \dots n\}$$

$y_i = 0 \longrightarrow \tilde{y}_i^{(0)} = 0 + \epsilon$

$y_i = 1 \longrightarrow \tilde{y}_i^{(1)} = 1 - \epsilon$

Small Number

Example: $\epsilon = 0.001$

$\tilde{y}_i^{(0)} = 0.001$

$\tilde{y}_i^{(1)} = 0.999$

Perceptron: Learning by Pseudo-Inverse

$$\tilde{y}_i = \frac{1}{1 + e^{-u_i}} \Rightarrow e^{-u_i} = \frac{1}{\tilde{y}_i} - 1 \Rightarrow e^{u_i} = \frac{\tilde{y}_i}{1 - \tilde{y}_i}$$

$$u_i = \log_e\left(\frac{\tilde{y}_i}{1 - \tilde{y}_i}\right)$$

$$u_i = \boldsymbol{\omega}^T \boldsymbol{x}_i + b$$

$$\boldsymbol{\omega}^T \boldsymbol{x}_i + b = \log_e\left(\frac{\tilde{y}_i}{1 - \tilde{y}_i}\right)$$

Perceptron: Learning by Pseudo-Inverse

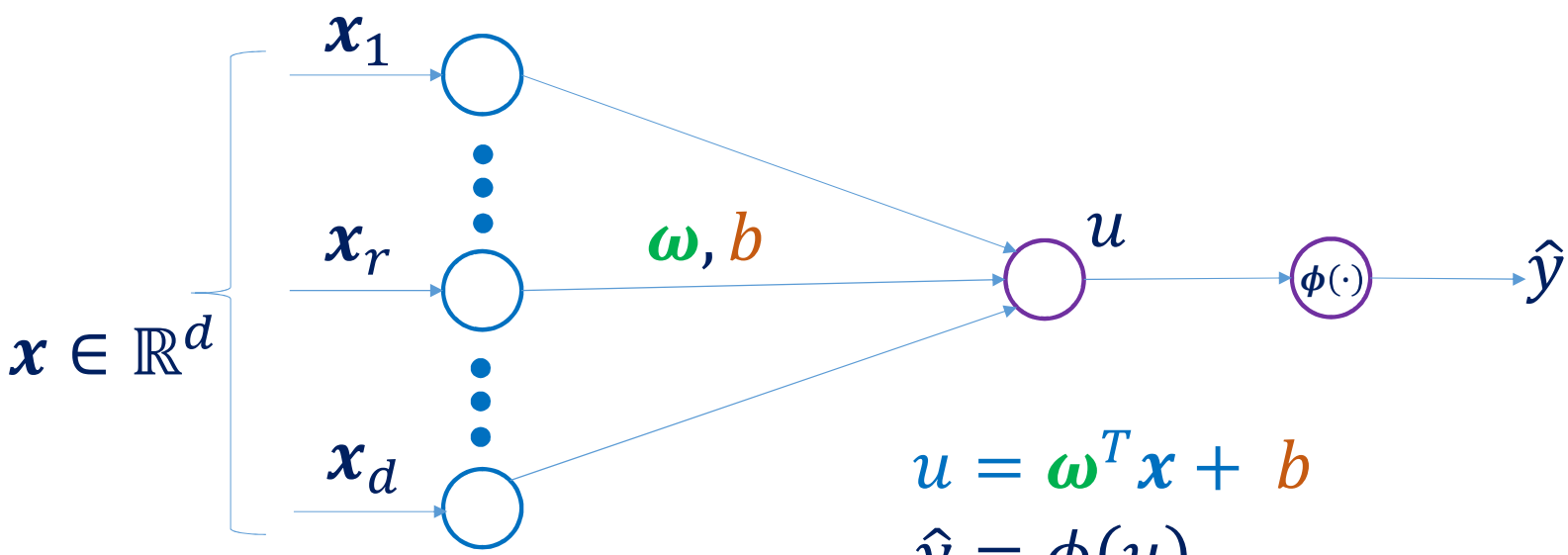
$$A = \begin{bmatrix} \boldsymbol{x}_1^T & 1 \\ \vdots & \vdots \\ \boldsymbol{x}_i^T & 1 \\ \vdots & \vdots \\ \boldsymbol{x}_n^T & 1 \end{bmatrix}_{n\times(d+1)}$$

$$P = \begin{bmatrix} \boldsymbol{\omega} \\ b \end{bmatrix}_{(d+1)\times 1}$$

$$B = \begin{bmatrix} \log_e\left(\frac{\tilde{y}_1}{1-\tilde{y}_1}\right) \\ \vdots \\ \log_e\left(\frac{\tilde{y}_i}{1-\tilde{y}_i}\right) \\ \vdots \\ \log_e\left(\frac{\tilde{y}_n}{1-\tilde{y}_n}\right) \end{bmatrix}_{n\times 1}$$

$$P = (A^T A)^{-1} A^T B = A^{\#} B$$

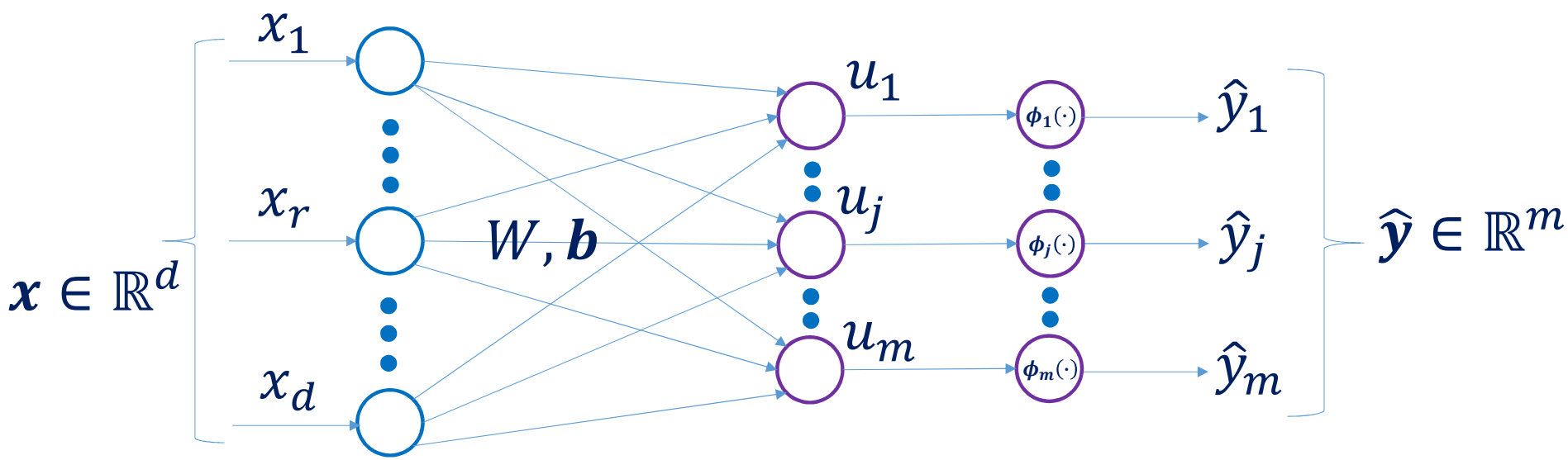
Perceptron: Vector Input, Scalar Output



$$u = \boldsymbol{\omega}^T \boldsymbol{x} + b$$

$$\hat{y} = \phi(u)$$

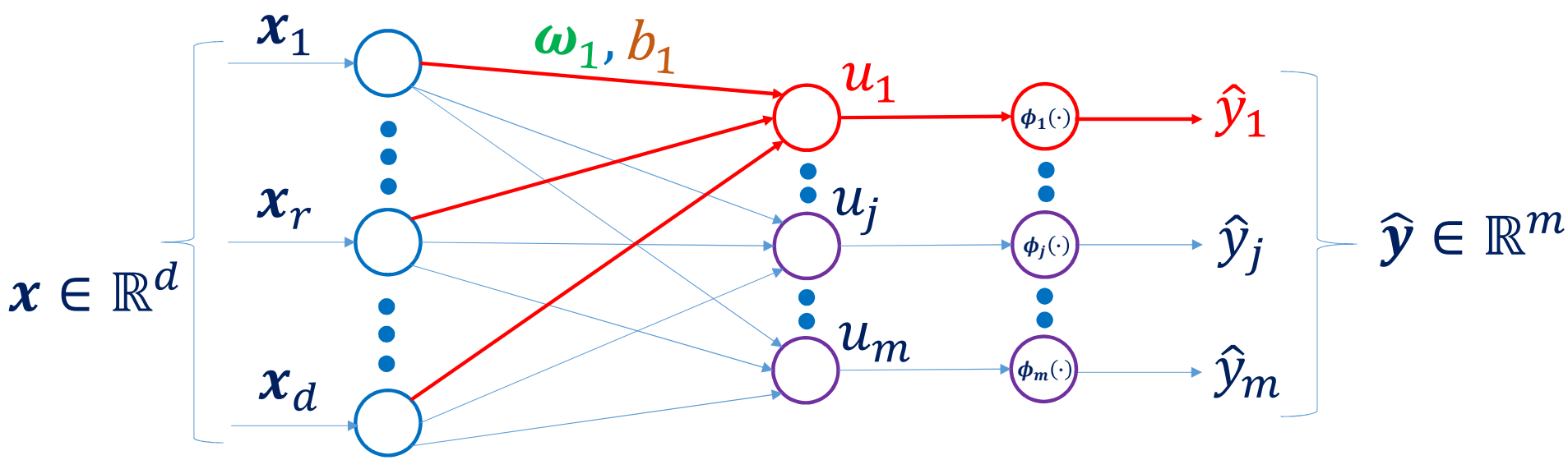
Perceptron: Vector Input, Vector Output



$$\hat{y}_j = \phi_j(u_j)$$

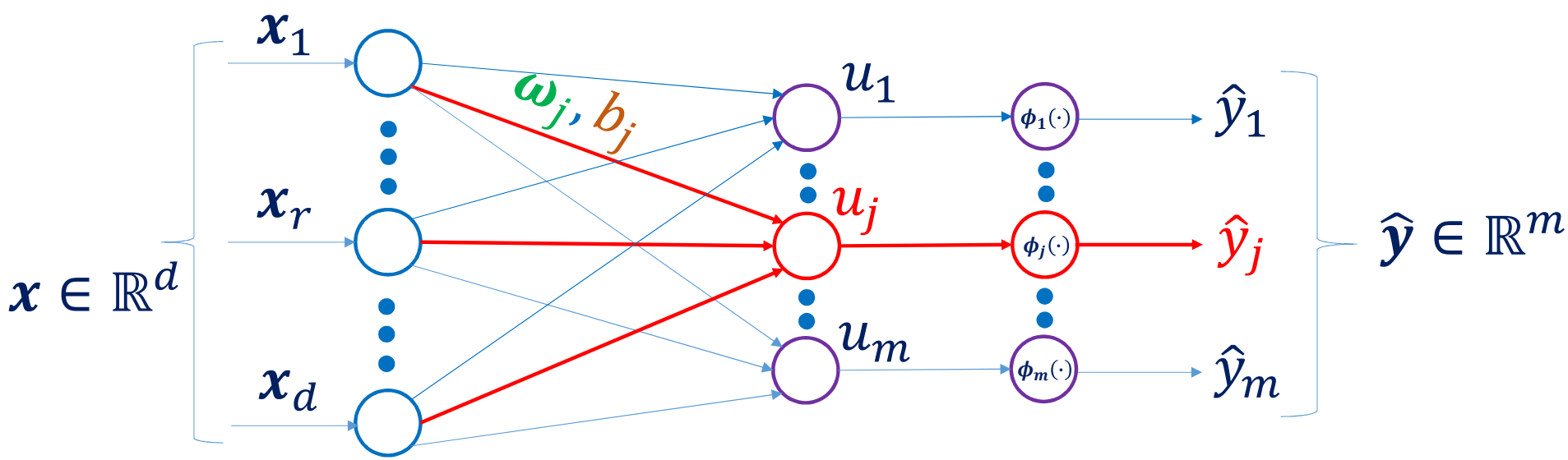
$$\boldsymbol{u} = \begin{bmatrix} u_1 \\ \vdots \\ u_j \\ \vdots \\ u_m \end{bmatrix} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

Perceptron: Vector Input, Vector Output



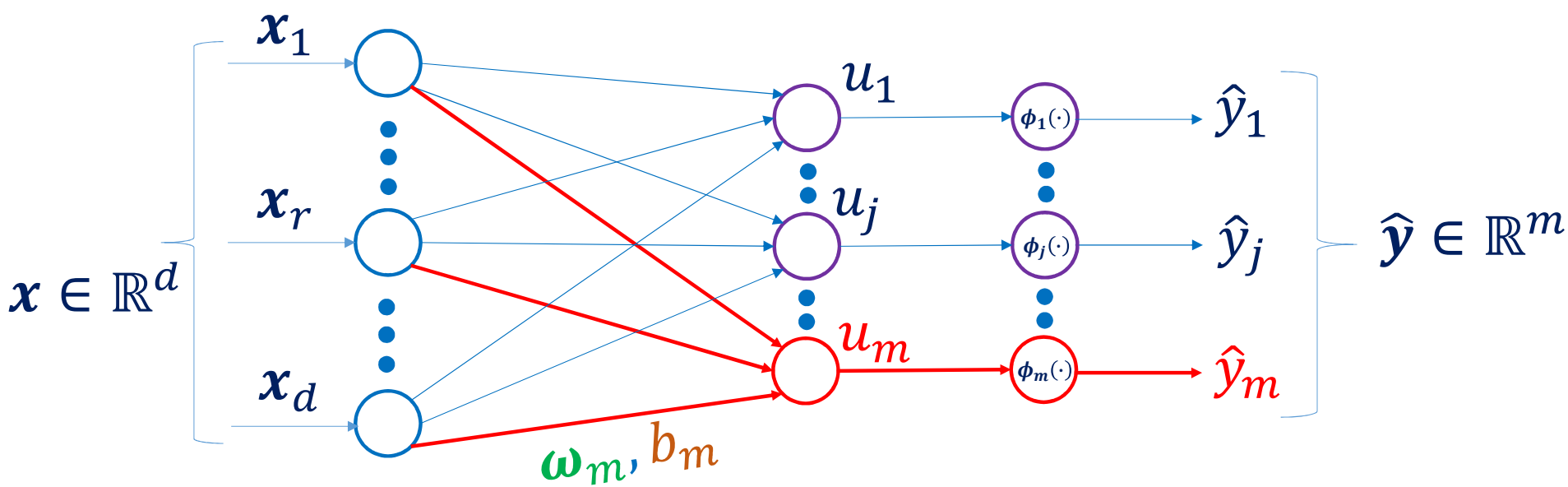
$$u_1 = \boldsymbol{\omega}_1^T \boldsymbol{x} + b_1 \qquad \hat{y}_1 = \phi_1(u_1)$$

Perceptron: Vector Input, Vector Output



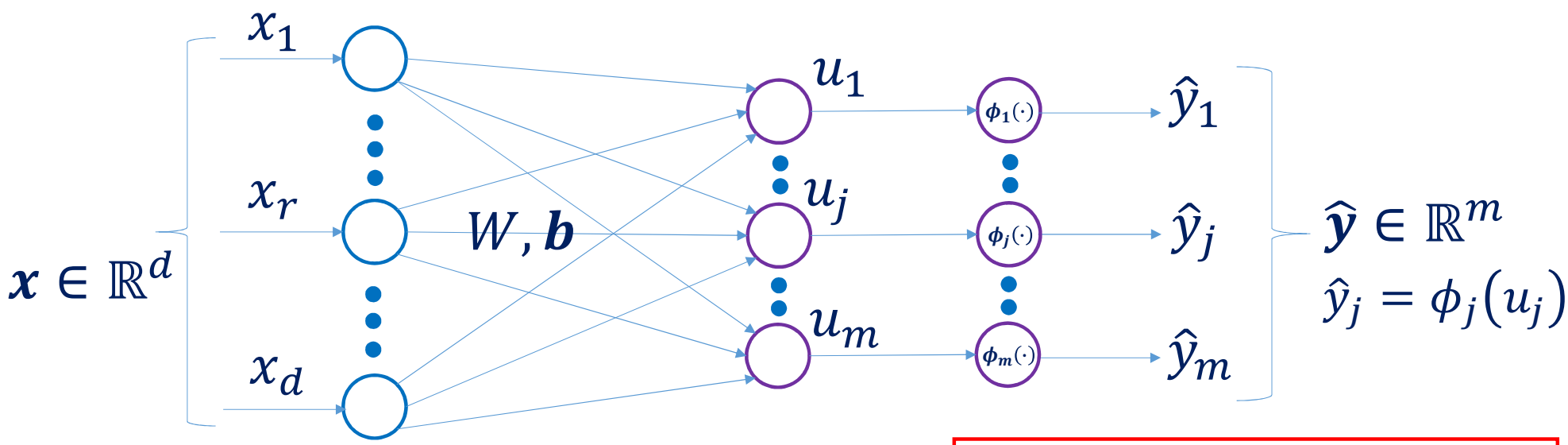
$$u_j = \boldsymbol{\omega}_j^T \boldsymbol{x} + b_j \qquad \hat{y}_j = \phi_j(u_j)$$

Perceptron: Vector Input, Vector Output



$$u_m = \boldsymbol{\omega}_m^T \boldsymbol{x} + b_m \qquad \hat{y}_m = \phi_m(u_m)$$

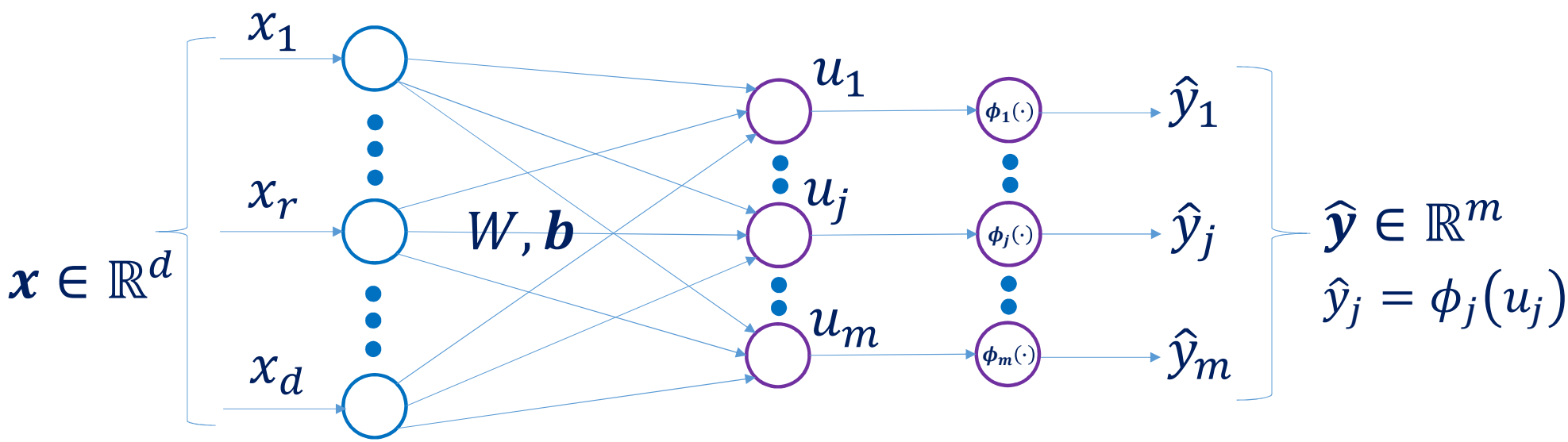
Perceptron: Vector Input, Vector Output



$$\boldsymbol{u} = \begin{bmatrix} u_1 \\ \vdots \\ u_j \\ \vdots \\ u_m \end{bmatrix} = \begin{bmatrix} \boldsymbol{\omega}_1^T \\ \vdots \\ \boldsymbol{\omega}_j^T \\ \vdots \\ \boldsymbol{\omega}_m^T \end{bmatrix} \boldsymbol{x} + \begin{bmatrix} b_1 \\ \vdots \\ b_j \\ \vdots \\ b_m \end{bmatrix} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

$$\boxed{\boldsymbol{W} = \begin{bmatrix} \boldsymbol{\omega}_1^T \\ \vdots \\ \boldsymbol{\omega}_j^T \\ \vdots \\ \boldsymbol{\omega}_m^T \end{bmatrix}_{m \times d} \qquad \boldsymbol{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_j \\ \vdots \\ b_m \end{bmatrix}_{m \times 1}}$$

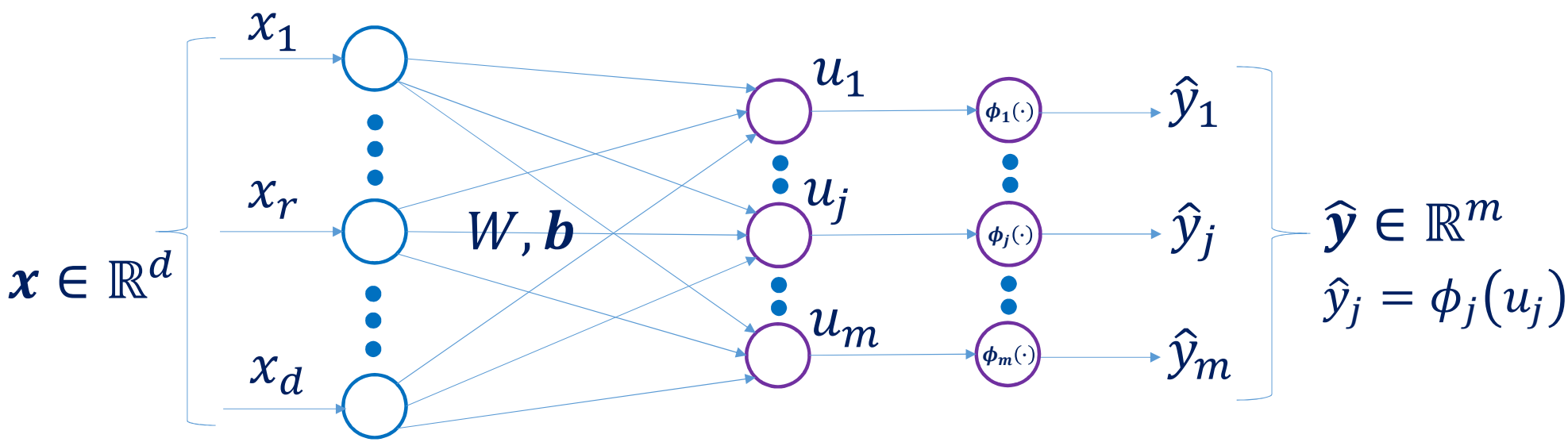
Perceptron: Vector Input, Vector Output



$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_r \\ \vdots \\ x_d \end{bmatrix} \quad \boldsymbol{W} = \begin{bmatrix} \boldsymbol{\omega}_1^T \\ \vdots \\ \boldsymbol{\omega}_j^T \\ \vdots \\ \boldsymbol{\omega}_m^T \end{bmatrix} \quad \boldsymbol{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_j \\ \vdots \\ b_m \end{bmatrix} \quad \hat{\boldsymbol{y}} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_j \\ \vdots \\ \hat{y}_m \end{bmatrix}$$

Interpret $\boldsymbol{W}_{kl} = \boldsymbol{W}[k][l]$

Perceptron: Vector Input, Vector Output



$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_r \\ \vdots \\ x_d \end{bmatrix} \quad \boldsymbol{W} = \begin{bmatrix} \boldsymbol{\omega}_1^T \\ \vdots \\ \boldsymbol{\omega}_j^T \\ \vdots \\ \boldsymbol{\omega}_m^T \end{bmatrix} \quad \boldsymbol{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_j \\ \vdots \\ b_m \end{bmatrix} \quad \hat{\boldsymbol{y}} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_j \\ \vdots \\ \hat{y}_m \end{bmatrix}$$

$\boldsymbol{W}_{kl} = \boldsymbol{W}[k][l]$ *is the Weight Value of the Edge Connecting Output Node* k *and Input Node* l

Perceptron Regression: Vector Function Approximation

Dataset: $\boldsymbol{S_R} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i); i = 1, \dots n\}$

$\boldsymbol{x} \in \mathbb{R}^d$

$y \in \mathbb{R}^m$

$$\boldsymbol{u}_i = W\boldsymbol{x}_i + \boldsymbol{b}$$

$$\widehat{\boldsymbol{y}}_{ij} = \phi(\boldsymbol{u}_i)$$

Instance Error: $e_{ij} = \boldsymbol{y}_{ij} - \widehat{\boldsymbol{y}}_{ij}$

Minimize Total Error: $E = \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{m} e_{ij}^2$

$$\boldsymbol{W}_{kl}^{(t+1)} = \boldsymbol{W}_{kl}^{(t)} - \eta_{kl}^{(t)} \frac{\partial E}{\partial \boldsymbol{W}_{kl}}$$

$$\boldsymbol{b}_k^{(t+1)} = \boldsymbol{b}_k^{(t)} - \eta_{kl}^{(t)} \frac{\partial E}{\partial \boldsymbol{b}_k}$$

Perceptron Regression: Vector Function Approximation

Dataset: $\boldsymbol{S_R} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i); i = 1, \dots n\}$

$\boldsymbol{x} \in \mathbb{R}^d$

$y \in \mathbb{R}^m$

$$\boldsymbol{u}_i = W\boldsymbol{x}_i + \boldsymbol{b}$$

$$\widehat{\boldsymbol{y}}_{ij} = \phi(\boldsymbol{u}_{ij})$$

Instance Error: $e_{ij} = \boldsymbol{y}_{ij} - \widehat{\boldsymbol{y}}_{ij}$

Minimize Total Error: $E = \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{m} e_{ij}^2$

$$\frac{\partial E}{\partial \boldsymbol{W}_{kl}} = ?$$

$$\frac{\partial E}{\partial \boldsymbol{b}_k} = ?$$

Assume Sigmoid Activation Function at Output

Perceptron Regression: Learning Parameters

$$E = \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{m} e_{ij}^2 = \frac{1}{n}\sum_{i=1}^{n}\left(e_{i1}^2 + e_{i2}^2 + \cdots + e_{ik}^2 + \cdots + e_{im}^2\right)$$

$$\frac{\partial E}{\partial \boldsymbol{W}_{kl}} = \frac{1}{n}\sum_{i=1}^{n}\left\{\frac{\partial\left(e_{i1}^2\right)}{\partial \boldsymbol{W}_{kl}} + \frac{\partial\left(e_{i2}^2\right)}{\partial \boldsymbol{W}_{kl}} + \cdots + \frac{\partial\left(e_{ik}^2\right)}{\partial \boldsymbol{W}_{kl}} + \cdots + \frac{\partial\left(e_{im}^2\right)}{\partial \boldsymbol{W}_{kl}}\right\}$$

$$\frac{\partial E}{\partial \boldsymbol{W}_{kl}} = \frac{1}{n}\sum_{i=1}^{n} 2e_{ik}\frac{\partial e_{ik}}{\partial \boldsymbol{W}_{kl}}$$

$\boldsymbol{W}_{kl} = \boldsymbol{W}[k][l]$ *is the Weight Value of the Edge Connecting Output Node* k *and Input Node* l

Perceptron Regression: Learning Parameters

$$\frac{\partial E}{\partial \boldsymbol{W}_{kl}} = \frac{1}{n}\sum_{i=1}^{n} 2e_{ik}\frac{\partial e_{ik}}{\partial \boldsymbol{W}_{kl}}$$

$$e_{ik} = \boldsymbol{y}_{ik} - \widehat{\boldsymbol{y}}_{ik}$$

$$\frac{\partial e_{ik}}{\partial \boldsymbol{W}_{kl}} = -\frac{\partial \widehat{\boldsymbol{y}}_{ik}}{\partial \boldsymbol{W}_{kl}}$$

$$\frac{\partial E}{\partial \boldsymbol{W}_{kl}} = -\frac{2}{n}\sum_{i=1}^{n} e_{ik}\frac{\partial \widehat{\boldsymbol{y}}_{ik}}{\partial \boldsymbol{W}_{kl}}$$

$$\frac{\partial \widehat{\boldsymbol{y}}_{ik}}{\partial \boldsymbol{W}_{kl}} = \frac{\partial \widehat{\boldsymbol{y}}_{ik}}{\partial \boldsymbol{u}_{ik}} \times \frac{\partial \boldsymbol{u}_{ik}}{\partial \boldsymbol{W}_{kl}}$$

Perceptron Regression: Learning Parameters

$$\hat{\boldsymbol{y}}_{ik} = \phi(\boldsymbol{u}_{ik}) = \frac{1}{1 + e^{-\boldsymbol{u}_{ik}}} \quad \Longrightarrow \quad \frac{\partial \hat{\boldsymbol{y}}_{ik}}{\partial \boldsymbol{u}_{ik}} = \phi'(\boldsymbol{u}_{ik}) = \hat{\boldsymbol{y}}_{ik}(1 - \hat{\boldsymbol{y}}_{ik})$$

$$\boldsymbol{u}_{ik} = \sum_{r=1}^{d} \boldsymbol{W}_{kr}\boldsymbol{x}_{ir} + \boldsymbol{b}_k = (\boldsymbol{W}_{k1}\boldsymbol{x}_{i1} + \cdots + \boldsymbol{W}_{kl}\boldsymbol{x}_{il} + \cdots \boldsymbol{W}_{kd}\boldsymbol{x}_{id}) + \boldsymbol{b}_{\mathrm{k}}$$

$$\frac{\partial \boldsymbol{u}_{ik}}{\partial \boldsymbol{W}_{kl}} = \boldsymbol{x}_{il}$$

$$\frac{\partial \hat{\boldsymbol{y}}_{ik}}{\partial \boldsymbol{W}_{kl}} = \frac{\partial \hat{\boldsymbol{y}}_{ik}}{\partial \boldsymbol{u}_{ik}} \times \frac{\partial \boldsymbol{u}_{ik}}{\partial \boldsymbol{W}_{kl}} = \hat{\boldsymbol{y}}_{ik}(1 - \hat{\boldsymbol{y}}_{ik})$$

Perceptron Regression: Learning Parameters

$$\frac{\partial E}{\partial \boldsymbol{W}_{kl}} = -\frac{2}{n}\sum_{i=1}^{n} e_{ik}\widehat{\boldsymbol{y}}_{ik}(1-\widehat{\boldsymbol{y}}_{ik})\,\boldsymbol{x}_{il}$$

$$\frac{\partial E}{\partial \boldsymbol{b}_{k}} = -\frac{2}{n}\sum_{i=1}^{n} e_{ik}\widehat{\boldsymbol{y}}_{ik}(1-\widehat{\boldsymbol{y}}_{ik})$$

Perceptron: Multicategory Classification

$$\boldsymbol{S_m} = \{(\boldsymbol{x}_i, y_i); i = 1, \dots n\}$$

$$\boldsymbol{x} \in \mathbb{R}^d$$

$$y \in \boldsymbol{Y}_L = \{1, \dots m\}$$

Category Label Set

$\boldsymbol{x}$ → Classifier $\boldsymbol{C_m}$ → $\hat{\boldsymbol{y}} = \boldsymbol{C_m}(\boldsymbol{x})$

$$\sum_{j=1}^{m} \hat{\boldsymbol{y}}_j = 1$$

$\hat{y}_j$ is an Estimate of the Posterior $P(\boldsymbol{\omega}_j|\boldsymbol{x})$ for Category j

Multi-Category Classification: Target Specification

$$\boldsymbol{S_m} = \{(\boldsymbol{x}_i, y_i); i = 1, \dots n\}$$

$\boldsymbol{x} \in \mathbb{R}^d$

$y \in \{1, \dots m\}$

One Hot Output Encoding

$$\tilde{\boldsymbol{y}}_{ij} = \tilde{\boldsymbol{y}}_i[j] = \begin{cases} 1, & y_i = j \\ 0, & y_i \neq j \end{cases}$$

$j = 1, \dots m$

$$\sum_{j=1}^{m} \tilde{\boldsymbol{y}}_{ij} = 1$$

$y_k = 1$

$y_r = 2$

$y_l = 3$

$$\tilde{\boldsymbol{y}}_k = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \tilde{\boldsymbol{y}}_r = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \tilde{\boldsymbol{y}}_l = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Soft-Max Activation Function

$$\hat{\boldsymbol{y}}_{ij} = \frac{e^{\boldsymbol{u}_{ij}}}{\sum_{r=1}^{m} e^{\boldsymbol{u}_{ir}}}$$

Soft-Max

$$\hat{\boldsymbol{y}}_{ij} = \frac{1}{1 + \sum_{\substack{r=1 \\ r \neq j}}^{m} e^{(\boldsymbol{u}_{ir} - \boldsymbol{u}_{ij})}}$$

$$y_i = \frac{e^{u_i}}{1 + e^{u_i}}$$

Sigmoid

$$y_i = \frac{1}{1 + e^{-u_i}}$$

Multi-Class Perceptron: Learning by Gradient Descent

$$\boldsymbol{S_m} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i); i = 1, \dots n\}$$

$$\boldsymbol{x} \in \mathbb{R}^d \qquad \boldsymbol{y} \in \{0,1\}^m$$

Cross-Entropy Loss

$$e_i = -\sum_{j=1}^{m} \widetilde{\boldsymbol{y}}_{ij} \log \widehat{\boldsymbol{y}}_{ij}$$

Total Loss

$$E = \frac{1}{n} \sum_{i=1}^{n} e_i$$

Multi-Class Perceptron: Learning by Gradient Descent

$$E = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{m}\widetilde{\boldsymbol{y}}_{ij}\log\widehat{\boldsymbol{y}}_{ij}$$

$$\boldsymbol{W}_{kl}^{(t+1)} = W_{kl}^{(t)} - \eta_{kl}^{(t)}\frac{\partial E}{\partial \boldsymbol{W}_{kl}}$$

$$\boldsymbol{b}_{k}^{(t+1)} = \boldsymbol{b}_{k}^{(t)} - \eta_{k}^{(t)}\frac{\partial E}{\partial \boldsymbol{b}_{k}}$$

Multi-Class Perceptron: Learning by Gradient Descent

$$E = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{m}\widetilde{\boldsymbol{y}}_{ij}\log\widehat{\boldsymbol{y}}_{ij}$$

$$\Downarrow$$

$$\frac{\partial E}{\partial \boldsymbol{W}_{kl}} = -\frac{1}{n}\sum_{i=1}^{n}\widetilde{\boldsymbol{y}}_{ik}\frac{\partial(\log\widehat{\boldsymbol{y}}_{ik})}{\partial \boldsymbol{W}_{kl}}$$

Multi-Class Perceptron: Learning by Gradient Descent

$$\hat{\boldsymbol{y}}_{ik} = \frac{e^{\boldsymbol{u}_{ij}}}{\sum_{r=1}^{m} e^{\boldsymbol{u}_{ir}}} \longrightarrow \log(\hat{\boldsymbol{y}}_{ik}) = \boldsymbol{u}_{ik} - \log\left(\sum_{r=1}^{m} e^{\boldsymbol{u}_{ir}}\right)$$

$$\frac{\partial(\log \hat{\boldsymbol{y}}_{ik})}{\partial \boldsymbol{W}_{kl}} = \frac{\partial \boldsymbol{u}_{ik}}{\partial \boldsymbol{W}_{kl}} - \frac{\partial\{\log(\sum_{r=1}^{m} e^{\boldsymbol{u}_{ir}})\}}{\partial \boldsymbol{W}_{kl}}$$

Multi-Class Perceptron: Learning by Gradient Descent

$$\frac{\partial\{\log(\sum_{r=1}^{m} e^{\boldsymbol{u}_{ir}})\}}{\partial \boldsymbol{W}_{kl}} = \frac{1}{\sum_{r=1}^{m} e^{u_{ir}}} \frac{\partial\{\sum_{r=1}^{m} e^{u_{ir}}\}}{\partial \boldsymbol{W}_{kl}} = \frac{1}{\sum_{r=1}^{m} e^{\boldsymbol{u}_{ir}}} \frac{\partial\{e^{\boldsymbol{u}_{ik}}\}}{\partial W_{kl}}$$

$$\Downarrow$$

$$\frac{\partial\{\log(\sum_{r=1}^{m} e^{\boldsymbol{u}_{ir}})\}}{\partial \boldsymbol{W}_{kl}} = \frac{e^{\boldsymbol{u}_{ik}}}{\sum_{r=1}^{m} e^{u_{ir}}} \frac{\partial\{\boldsymbol{u}_{ik}\}}{\partial \boldsymbol{W}_{kl}} = \widehat{\boldsymbol{y}}_{ij} \frac{\partial\{\boldsymbol{u}_{ik}\}}{\partial \boldsymbol{W}_{kl}}$$

$$\Downarrow$$

$$\frac{\partial(\log \widehat{\boldsymbol{y}}_{ik})}{\partial \boldsymbol{W}_{kl}} = \frac{\partial \boldsymbol{u}_{ik}}{\partial \boldsymbol{W}_{kl}} - \widehat{\boldsymbol{y}}_{ik} \frac{\partial \boldsymbol{u}_{ik}}{\partial \boldsymbol{W}_{kl}} = (1 - \widehat{\boldsymbol{y}}_{ik}) \frac{\partial \boldsymbol{u}_{ik}}{\partial \boldsymbol{W}_{kl}}$$

Multi-Class Perceptron: Learning by Gradient Descent

$$\boldsymbol{u}_{ik} = \sum_{q=1}^{d} \boldsymbol{W}_{kq}\boldsymbol{x}_{iq} + \boldsymbol{b}_k \quad \Longrightarrow \quad \frac{\partial \boldsymbol{u}_{ik}}{\partial \boldsymbol{W}_{kl}} = \boldsymbol{x}_{il}$$

$$\frac{\partial(\log \widehat{\boldsymbol{y}}_{ik})}{\partial \boldsymbol{W}_{kl}} = (1 - \widehat{\boldsymbol{y}}_{ik})\boldsymbol{x}_{il}$$

$$\frac{\partial E}{\partial \boldsymbol{W}_{kl}} = -\frac{1}{n}\sum_{i=1}^{n} \widetilde{\boldsymbol{y}}_{ik} \frac{\partial(\log \widehat{\boldsymbol{y}}_{ik})}{\partial \boldsymbol{W}_{kl}} = -\frac{1}{n}\sum_{i=1}^{n} \widetilde{\boldsymbol{y}}_{ik}(1 - \widehat{\boldsymbol{y}}_{ik})\boldsymbol{x}_{il}$$

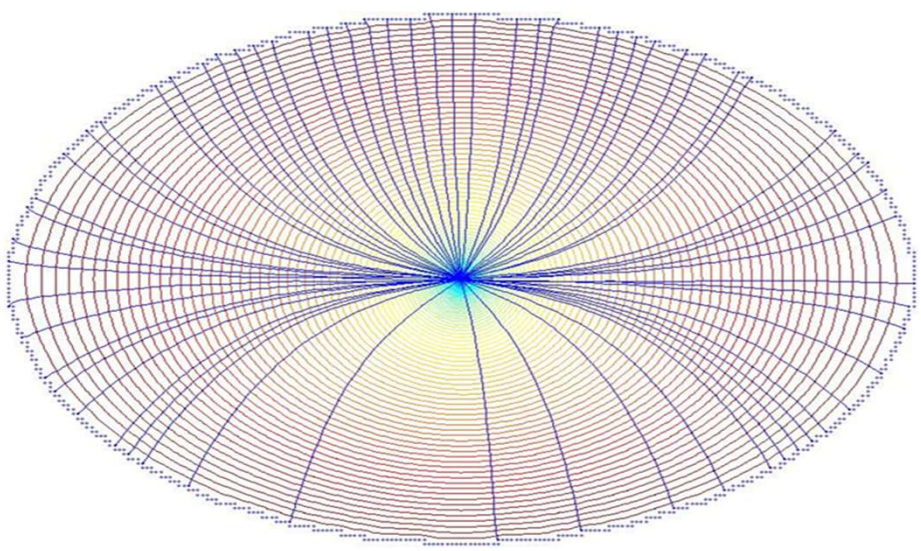
Multi-Class Perceptron: Learning by Gradient Descent

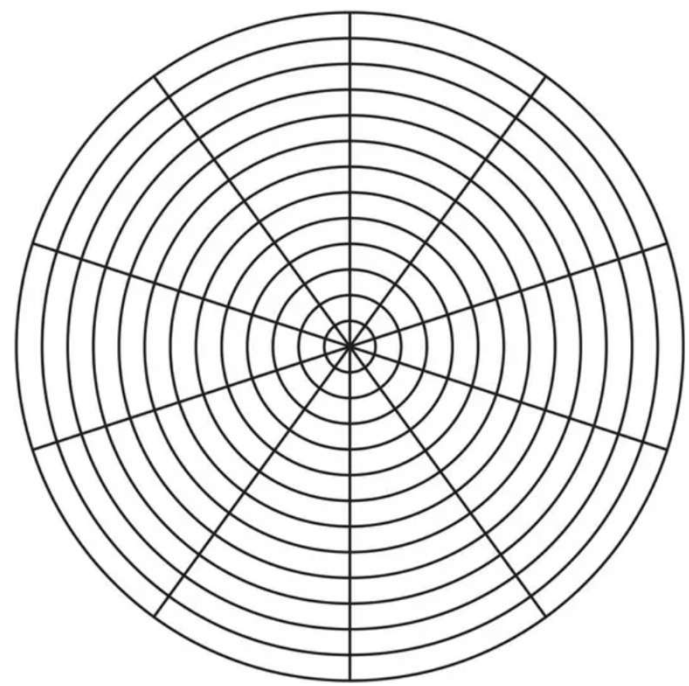
$$\frac{\partial E}{\partial \boldsymbol{W}_{kl}} = -\frac{1}{n}\sum_{i=1}^{n} \widetilde{\boldsymbol{y}}_{ik}(1-\widehat{\boldsymbol{y}}_{ik})\boldsymbol{x}_{il}$$

$$\frac{\partial E}{\partial \boldsymbol{b}_{k}} = -\frac{1}{n}\sum_{i=1}^{n} \widetilde{\boldsymbol{y}}_{ik}(1-\widehat{\boldsymbol{y}}_{ik})$$

Perceptron: Input Normalization

$$\left(\frac{x - c_x}{s_x}\right)^2 + \left(\frac{y - c_y}{s_y}\right)^2 = 1$$





$$(x - c_x)^2 + \left(y - c_y\right)^2 = r^2$$

Perceptron: Ideal Input Normalization

$$S_{trn} = \{(\boldsymbol{x}_i, y_i); i = 1, \dots n;\}$$

$$C = Cov(X)$$ Covariance Matrix

$$\mu = Mean(X)$$ Mean Vector

Matrix of Eigen Values of Covariance Matrix

$$C = Q\hat{C}Q^T$$

Matrix of Eigen Vectors of Covariance Matrix

$$\boxed{\tilde{\boldsymbol{x}}_i = \hat{C}^{-1}Q^T(\boldsymbol{x}_i - \boldsymbol{\mu})}$$

Zero-Centering $\longrightarrow$ Axis Aligning $\longrightarrow$ Scaling

Perceptron: Input Normalization

Mean Value Along Dimension ***r***

$$\tilde{\boldsymbol{x}}_i[r] = \frac{\boldsymbol{x}_i[r] - \boldsymbol{\mu}[r]}{\boldsymbol{\sigma}[r]}$$

Standard Deviation Along Dimension ***r***

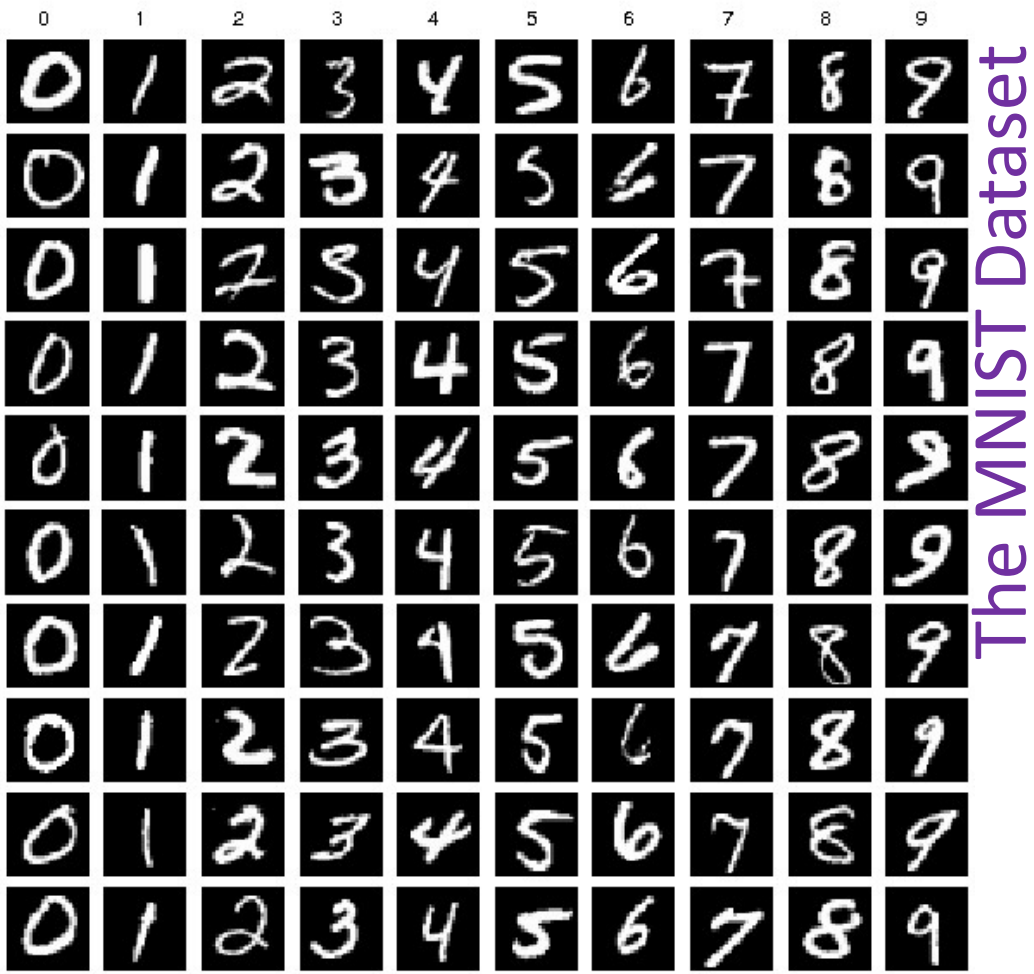
Perceptron: Input Normalization

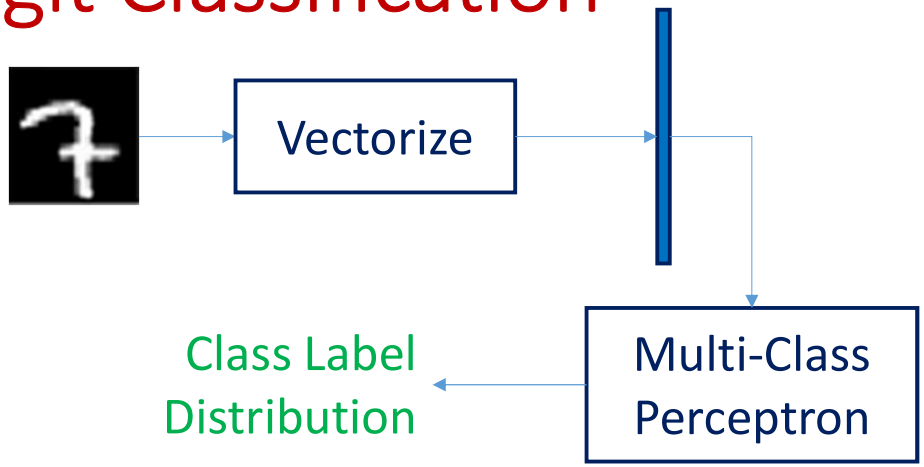
$$\tilde{\boldsymbol{x}}_i[r] = \frac{\boldsymbol{x}_i[r] - \boldsymbol{x}_{min}[r]}{\boldsymbol{x}_{max}[r] - \boldsymbol{x}_{min}[r]}$$

Minimum Value Along Dimension ***r***

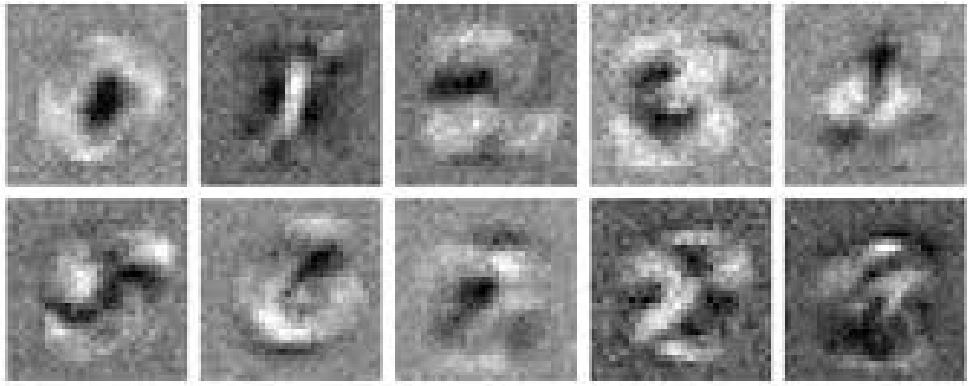
Maximum Value Along Dimension ***r***

Application to Handwritten Digit Classification





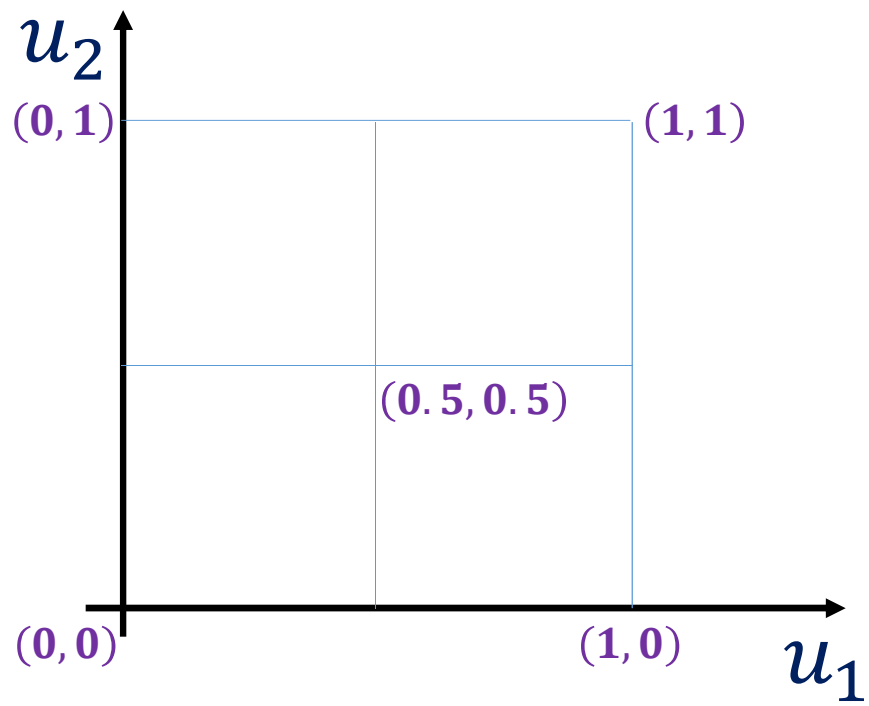
Weight Visualization



Dataset for Logical Operations

$$x_1 = \begin{cases} 1, & u_1 \in [0,0.5) \\ 0, & u_1 \in [0.5,1] \end{cases}$$

$$x_2 = \begin{cases} 1, & u_2 \in [0,0.5) \\ 0, & u_2 \in [0.5,1] \end{cases}$$



Two-Input-Single-Output Logical Operations

Logical OR		
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

$$y = x_1 \vee x_2$$

$$z = u_1 + u_2$$

Logical AND		
x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

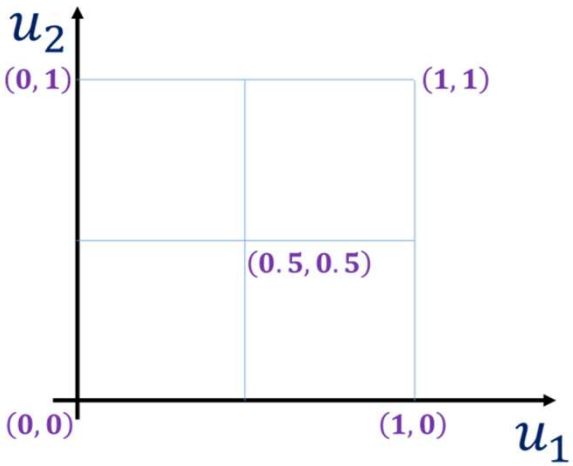
$$y = x_1 \wedge x_2$$

$$z = u_1 u_2$$

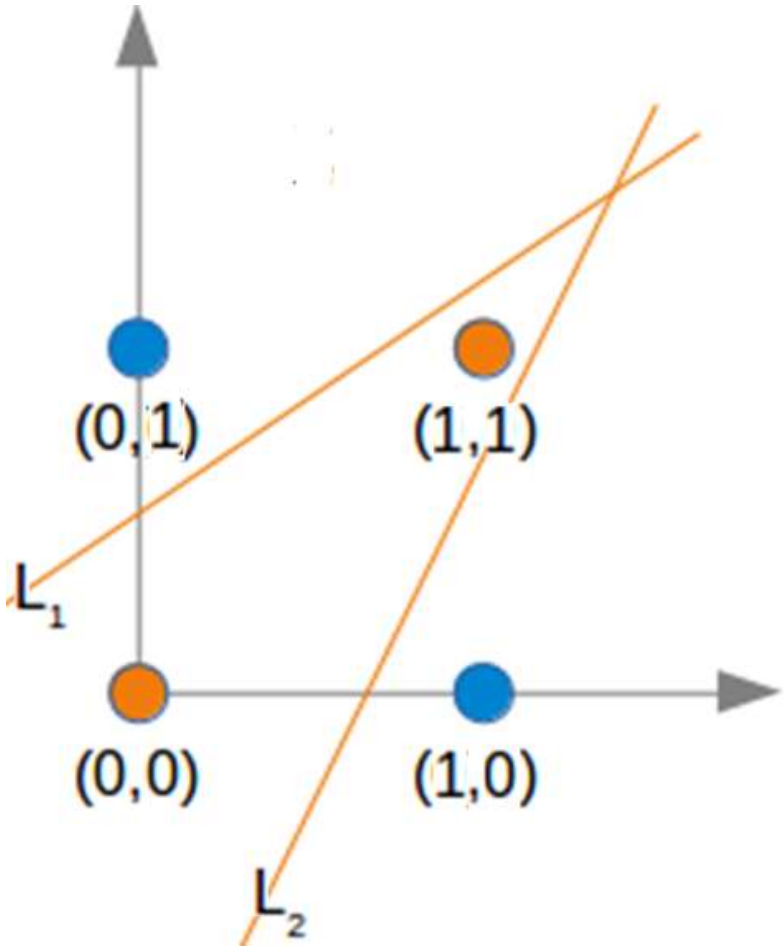
Logical XOR		
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

$$y = x_1 \oplus x_2$$

$$z = u_1(1 - u_2) + u_2(1 - u_1)$$



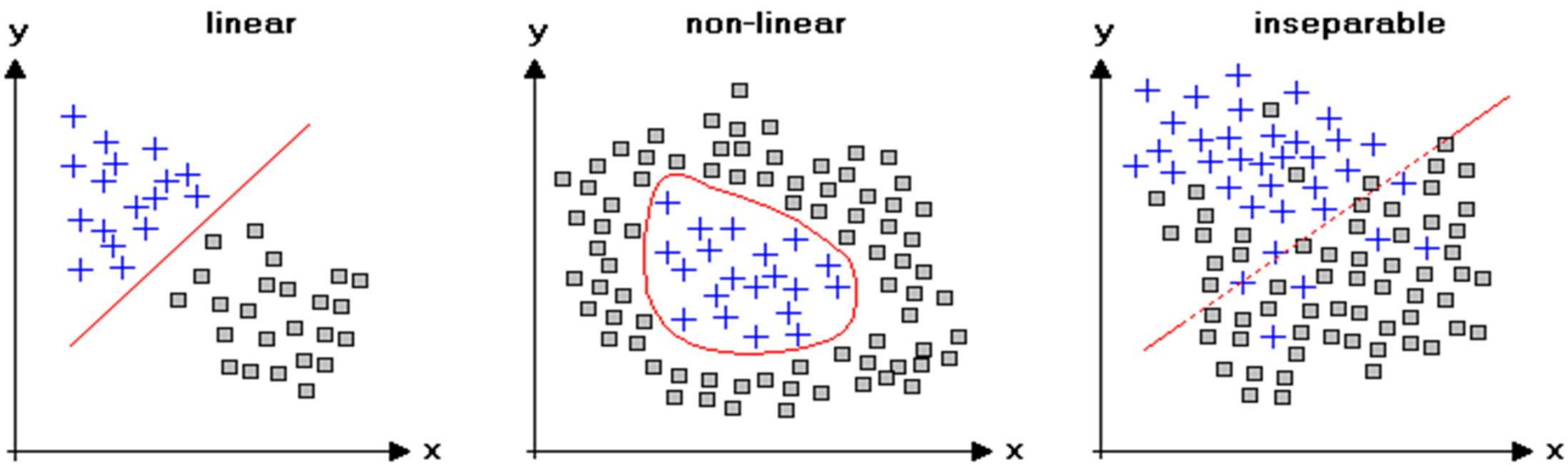
Perceptron: Failure Cases



The XOR Problem

$$y = x_1(1 - x_2) + x_2(1 - x_1)$$

Perceptron: Failure Cases



Covers Theorem of Separability

Nonlinear Transformation to a Higher Dimensional Space Increases the Probability of Linear Separability

Assignments

Perceptron: Two Category Classification

Dataset: $\boldsymbol{S_b} = \{(\boldsymbol{x}_i, y_i); i = 1, \dots n\}$

$\boldsymbol{x} \in \mathbb{R}^d$

$y \in \{0,1\}$

Instance Error: $e_i = -\{y_i log\hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\}$

$u_i = \boldsymbol{\omega}^T \boldsymbol{x}_i + b$

$\hat{y}_i = \dfrac{1}{1 + e^{-u_i}}$

Minimize Total Loss: $E = \frac{1}{n}\sum_{i=1}^{n} e_i$

$$\omega_j^{(k+1)} = \omega_j^{(k)} - \eta_{jk}\frac{\partial E}{\partial \boldsymbol{\omega}_j}$$

$$b^{(k+1)} = b^{(k)} - \eta_{jk}\frac{\partial E}{\partial b}$$

$j = 1, \dots d$

Evaluate:

$$\frac{\partial E}{\partial \boldsymbol{\omega}_j}, \frac{\partial E}{\partial \boldsymbol{b}}$$

A Three Category Classification Problem

Instance (i)	1	2	3	4	5	6	7	8	9	10
Ground Truth Label $(\boldsymbol{y_i})$	1	2	3	2	2	1	3	1	2	3
Prediction $(\widetilde{\boldsymbol{y}}_i[1])$	0.5	0.55	0.15	0.35	0.20	0.37	0.18	0.60	0.51	0.08
Prediction $(\widetilde{\boldsymbol{y}}_i[2])$	0.3	0.35	0.40	0.40	0.70	0.38	0.39	0.25	0.44	0.10
Prediction $(\widetilde{\boldsymbol{y}}_i[3])$	0.2	0.1	0.45	0.25	0.10	0.25	0.43	0.15	0.05	0.82

- Compute the Cross-Entropy Loss (averaged over entire dataset)
- Use Winner-Take-All (WTA) Strategy and Construct the Confusion Matrix
- Visualize Appearance of Ground-Truth in Ranked Scores
- Compute Accuracy and Comment on WTA Strategy

Summary

- Separability of Data in Feature Space
- Input-Output Description for Classification Problems
- Perceptron: Motivation & Formulation
- Perceptron: Interpretations
- Perceptron: Learning from Data
- Perceptron: Multi-category Classification
- Perceptron: Applications
- Failure Cases: Towards Multi-Layered Perceptron

Thank You